Informe de laboratorio 09

Tema: Práctica de laboratorio 09

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
09	Práctica de laboratorio 09	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	——	——

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa Nearest Neighbor para un TSP de 15 ciudades generadas aleatoriamente, y ejecútalo partiendo de cada una de las 15 ciudades como origen. ¿El resultado cambia según el punto de partida?

```cpp
#include "tsp.h"

int main() {
    const int N = 15;
    Instancia I = generar(N);

    vector<double> costos(N);
    printf("%-7s %10s\n", "Origen", "Costo NN");
    for (int o = 0; o < N; o++) {
        costos[o] = costo(vecinoCercano(o, I.d), I.d);
        printf("%-7d %10.3f\n", o, costos[o]);
    }

    Resumen r = resumir(costos);
    printf("\nMejor: %.3f (origen %d) | Peor: %.3f | Media: %.3f\n", r.mejor, r.idxMejor,
    ↪  r.peor, r.media);
    printf("Orgenes que alcanzan el mejor: %d/%d | Costos distintos: %d\n", r.enMejor, N,
    ↪  r.distintos);
```

```
    printf("¿Cambia según el origen? %s (peor excede al mejor en %.2f%%)\n",
           r.distintos > 1 ? "SÍ" : "NO", 100 * (r.peor - r.mejor) / r.mejor);
}
```

- /relleno

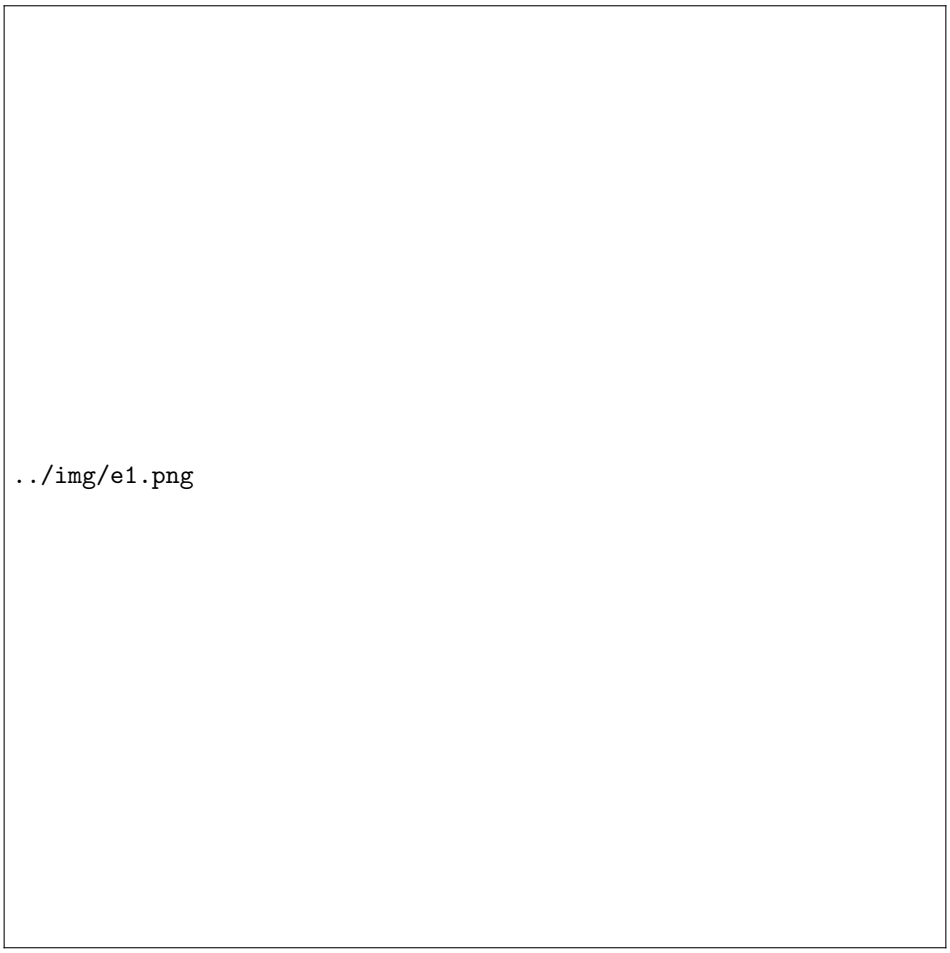


Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

Aplica 2-opt sobre las 15 soluciones del ejercicio anterior. ¿El mejor resultado final sigue dependiendo de la ciudad de origen usada por Nearest Neighbor?

```
// Ejercicio 2: 2-opt sobre las 15 soluciones de Nearest Neighbor (Ejercicio 1).
// Pregunta: ¿el mejor resultado final sigue dependiendo de la ciudad de origen?
#include "tsp.h"

int main() {
    const int N = 15;
    Instancia I = generar(N);  // misma semilla que el Ejercicio 1: mismas ciudades
```

```
8
9       vector<double> antes(N), despues(N);
10      printf("%-7s %10s %12s %9s\n", "Origen", "NN", "NN + 2-opt", "Mejora");
11      for (int o = 0; o < N; o++) {
12          Ruta nn = vecinoCercano(o, I.d);
13          antes[o] = costo(nn, I.d);
14          despues[o] = costo(dosOpt(nn, I.d), I.d);
15          printf("%-7d %10.3f %12.3f %8.2f%%\n", o, antes[o], despues[o], 100 * (antes[o] -
            ↪  despues[o]) / antes[o]);
16      }
17
18      Resumen a = resumir(antes), d = resumir(despues);
19      printf("\n%-11s %9s %9s %9s %12s %10s\n", "", "Mejor", "Peor", "Media", "Casos mejor",
        ↪  "Distintos");
20      printf("%-11s %9.3f %9.3f %9.3f %9d/%d %10d\n", "NN", a.mejor, a.peor, a.media,
        ↪  a.enMejor, N, a.distintos);
21      printf("%-11s %9.3f %9.3f %9.3f %9d/%d %10d\n", "NN + 2-opt", d.mejor, d.peor, d.media,
        ↪  d.enMejor, N, d.distintos);
22
23      printf("\nDispersión (peor vs mejor): NN %.2f%% -> NN + 2-opt %.2f%%\n",
24             100 * (a.peor - a.mejor) / a.mejor, 100 * (d.peor - d.mejor) / d.mejor);
25      printf("Origen del mejor: NN = %d, NN + 2-opt = %d\n", a.idxMejor, d.idxMejor);
26      printf("El resultado final depende del origen? %s (%d costos distintos; el mejor se
        ↪  alcanza desde %d de %d orgenes)\n",
27             d.distintos > 1 ? "SÍ" : "NO", d.distintos, d.enMejor, N);
28  }
```

- /relleno

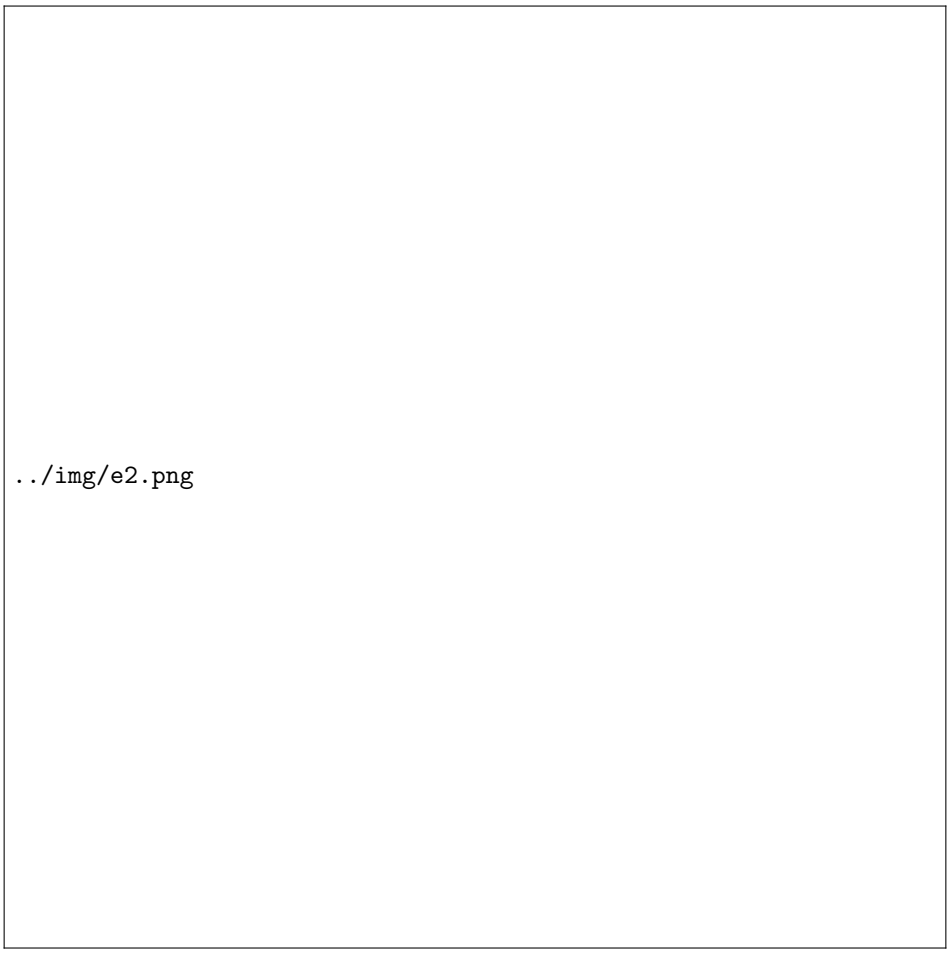


Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

Implementa otra heurística constructiva: la heurística del "vecino más barato"(cheapest insertion), que en cada paso inserta la ciudad no visitada que menos aumenta el costo total, en cualquier posición de la ruta. Compárala contra Nearest Neighbor.

```cpp
// Ejercicio 3: Inserción más barata (cheapest insertion) vs Nearest Neighbor.
// Ambas se ejecutan desde cada origen sobre las mismas 15 ciudades de los ejercicios 1 y 2.
#include "tsp.h"

// Tiempo medio (ms) de ejecutar la heurstica desde los N orgenes; se repite porque una sola corrida dura casi 0.
template <class F> double tiempoMs(F heuristica, const Mat& d, int reps = 2000) {
    auto t0 = chrono::steady_clock::now();
    double sumidero = 0;  // evita que el compilador elimine las llamadas
    for (int k = 0; k < reps; k++)
        for (int o = 0; o < (int)d.size(); o++) sumidero += heuristica(o, d)[0];
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count() / reps + 0 * sumidero;
}
```

```cpp

int main() {
    const int N = 15;
    Instancia I = generar(N);

    vector<double> nn(N), ci(N);
    int gCI = 0, gNN = 0, emp = 0;
    printf("%-7s %10s %12s %8s\n", "Origen", "NN", "Ins. barata", "Gana");
    for (int o = 0; o < N; o++) {
        nn[o] = costo(vecinoCercano(o, I.d), I.d);
        ci[o] = costo(insercionBarata(o, I.d), I.d);
        bool empate = fabs(nn[o] - ci[o]) < EPS;
        (empate ? emp : ci[o] < nn[o] ? gCI : gNN)++;
        printf("%-7d %10.3f %12.3f %8s\n", o, nn[o], ci[o], empate ? "empate" : ci[o] <
        ↪  nn[o] ? "CI" : "NN");
    }

    Resumen a = resumir(nn), b = resumir(ci);
    double tNN = tiempoMs(vecinoCercano, I.d), tCI = tiempoMs(insercionBarata, I.d);
    printf("\n%-12s %9s %9s %9s %12s %10s %11s\n", "", "Mejor", "Peor", "Media", "Casos
    ↪  mejor", "Distintos", "Tiempo(ms)");
    printf("%-12s %9.3f %9.3f %9.3f %9d/%d %10d %11.4f\n", "NN", a.mejor, a.peor, a.media,
    ↪  a.enMejor, N, a.distintos, tNN);
    printf("%-12s %9.3f %9.3f %9.3f %9d/%d %10d %11.4f\n", "Ins. barata", b.mejor, b.peor,
    ↪  b.media, b.enMejor, N, b.distintos, tCI);

    printf("\nPor origen: CI gana %d, NN gana %d, empates %d\n", gCI, gNN, emp);
    printf("Mejora media de CI sobre NN: %.2f%% | Mejor global: %s\n", 100 * (a.media -
    ↪  b.media) / a.media,
           fabs(a.mejor - b.mejor) < EPS ? "empate" : a.mejor < b.mejor ? "NN" : "CI");
    printf("Costo de tiempo: CI es %.1f veces más lenta que NN\n", tCI / tNN);
}
```

- /relleno

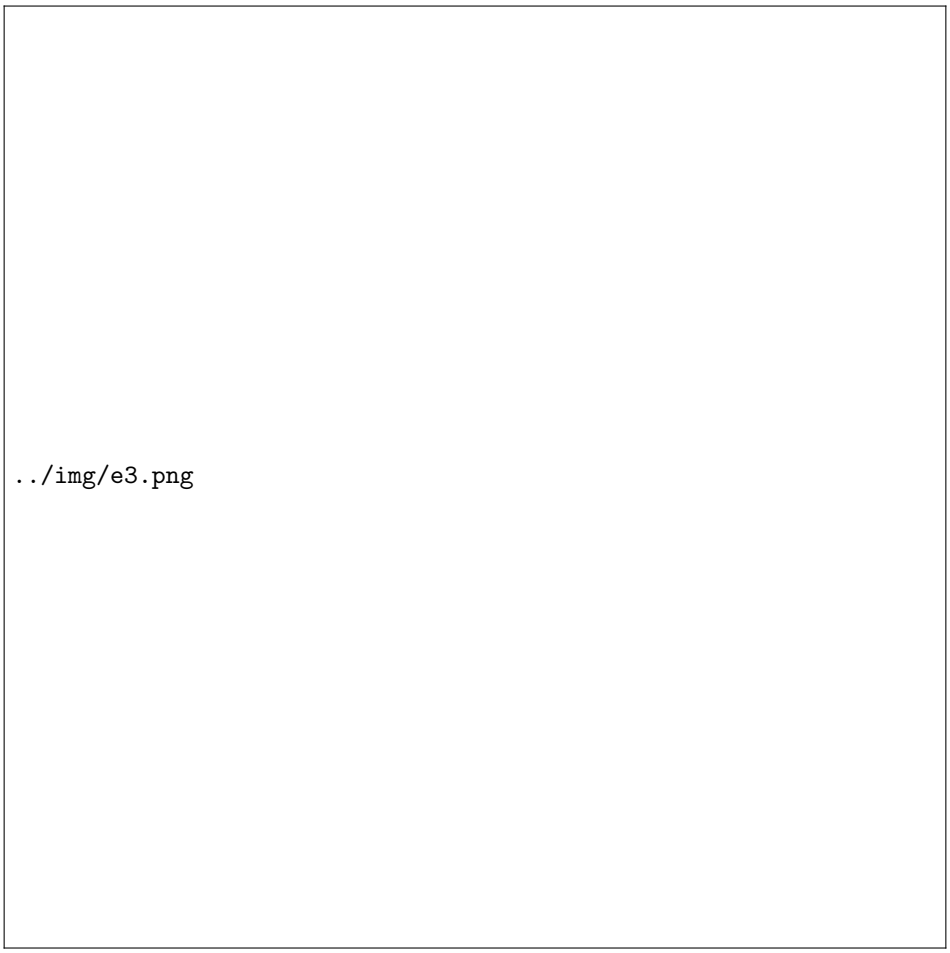


Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

Mide el tiempo de ejecución de Nearest Neighbor + 2-opt para instancias de 20, 50 y 100 ciudades, y compáralo contra el tiempo que tomaría fuerza bruta para esos mismos tamaños (estimado, sin ejecutarlo).

```
1
```

- /relleno

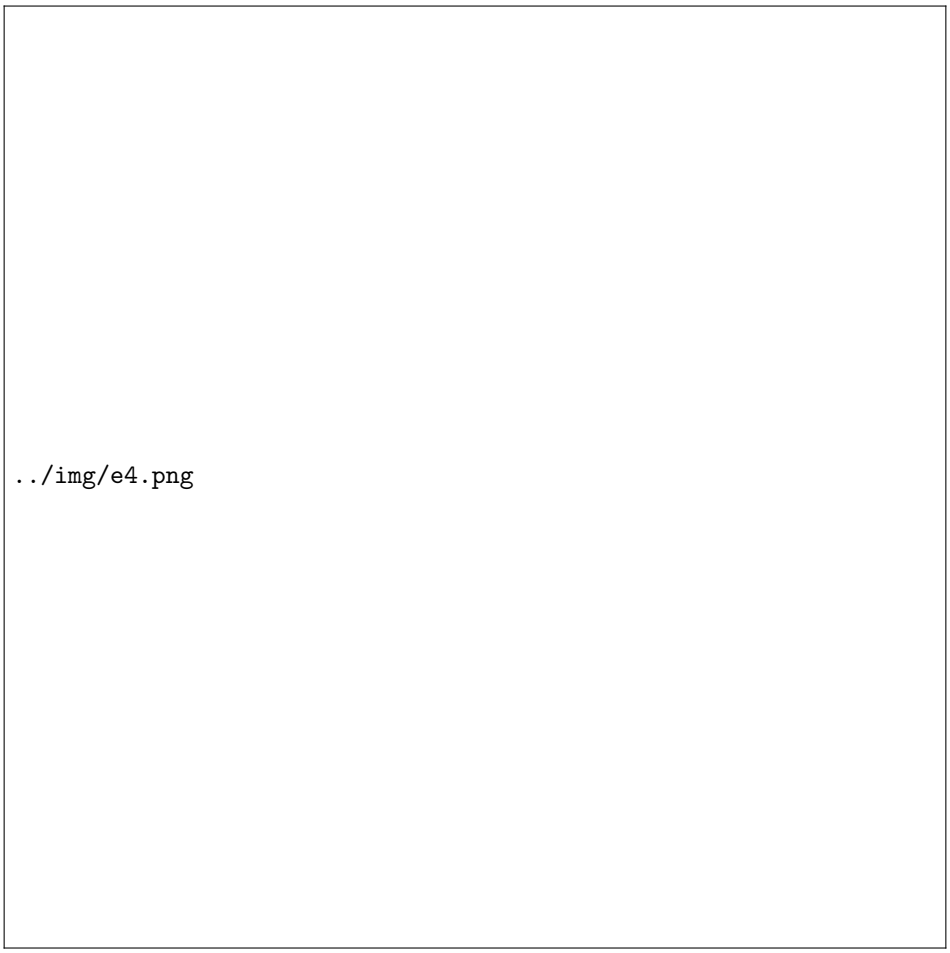


Figura 4: Ejecución del ejercicio 4

1.5. Ejercicio 5

Aplica Nearest Neighbor + 2-opt a un caso real: una lista de direcciones o puntos de interés de tu ciudad (puedes usar coordenadas aproximadas), y calcula la ruta de reparto o visita más corta posible.

```
1
```

- /relleno

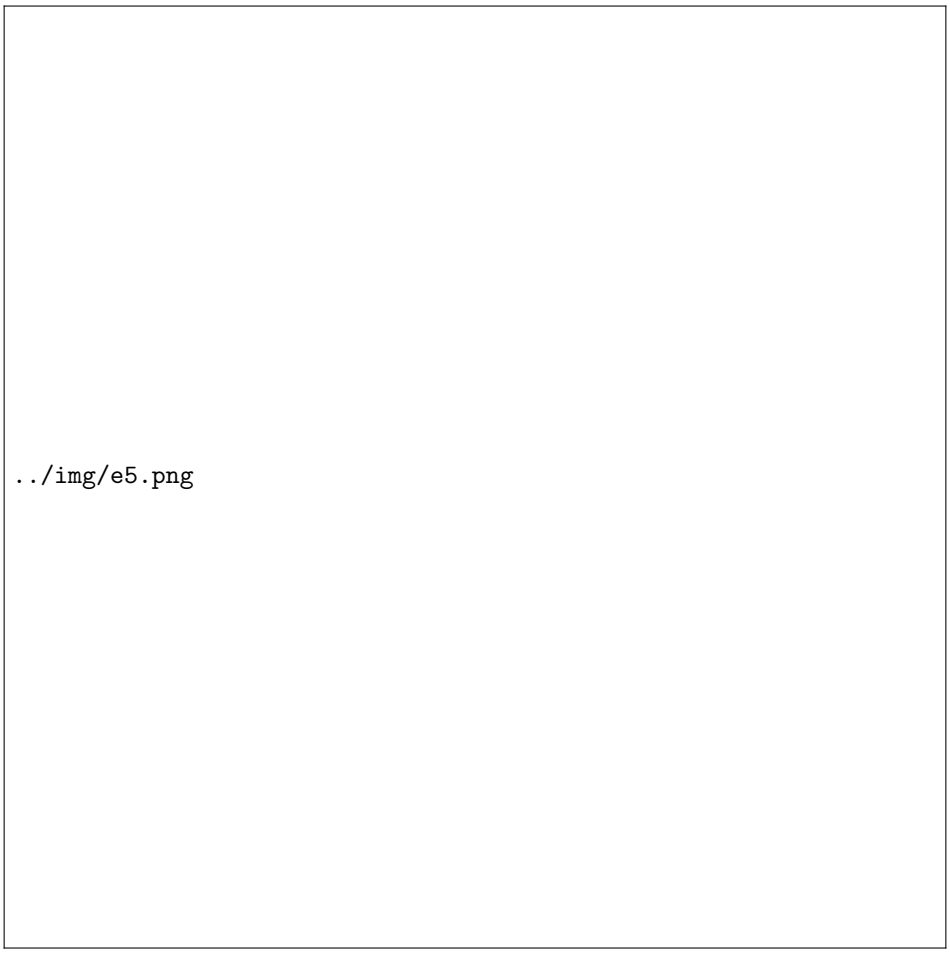


Figura 5: Ejecución del ejercicio 5

2. Cuestionario

- ¿Cuál es la diferencia entre un método aproximado y un método heurístico?
- ¿Por qué Nearest Neighbor puede terminar con un salto muy costoso al final de la ruta? Usa como ejemplo el inicio en la ciudad 2 de la Tabla 3.
- ¿Qué significa que 2-opt encuentre un "óptimo local¿ ¿Por qué no garantiza el óptimo global?
- ¿Por qué combinar una heurística constructiva con una de búsqueda local suele dar mejores resultados que usar cualquiera de las dos por separado?
- Comparando la Figura 5 (y la Tabla 5) de este laboratorio con lo visto en el Laboratorio 8, ¿por qué Nearest Neighbor + 2-opt es preferible a Monte Carlo incluso con un presupuesto de cómputo similar?

3. Conclusiones